

Embedded AI/ML Development with ESP32

Dr. Hiran Ekanayake

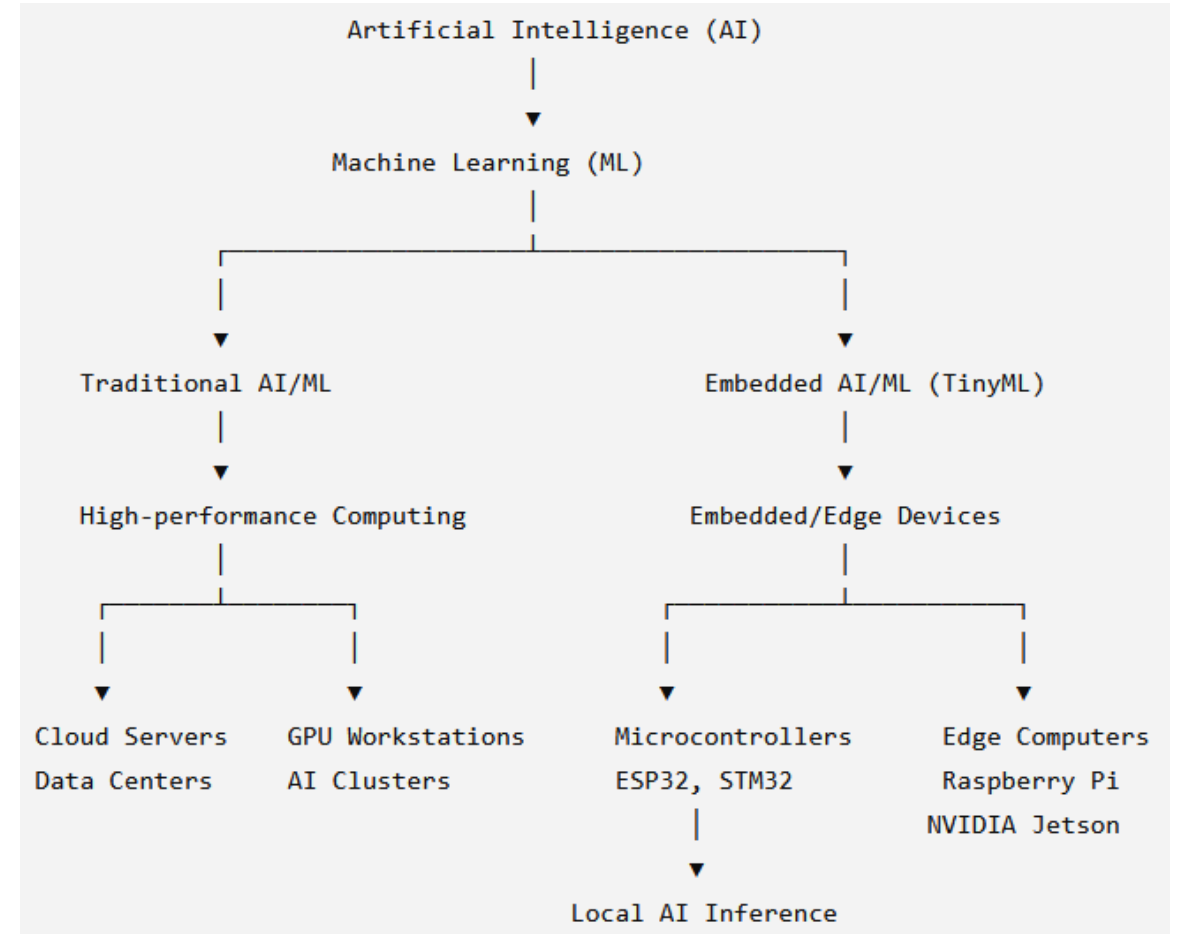
University of Colombo School of Computing

2026

Part 1

What is Embedded AI/ML?

- Embedded AI combines three fields:
 - Embedded Systems
 - Artificial Intelligence
 - Machine Learning
- The AI model is stored in the embedded device's memory and runs locally using sensor data.
- Traditional AI/ML Architecture:
 - The embedded device mainly collects data and sends it to the cloud, where AI processing occurs.



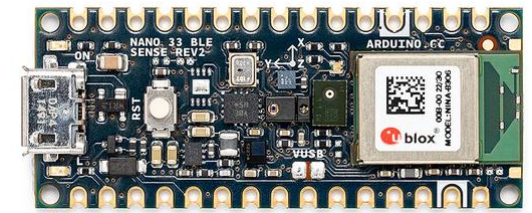
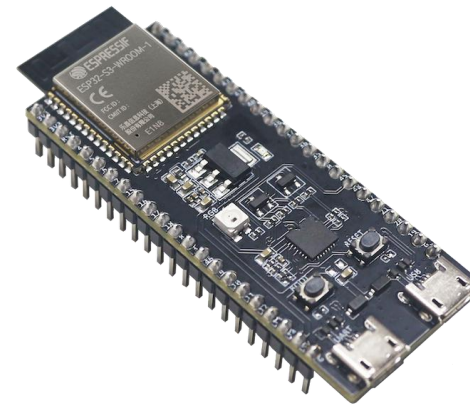
Key Differences

Feature	Embedded AI/ML	Traditional AI/ML
Processing Location	Device (Edge)	Cloud or Server
Internet Requirement	Usually not required	Usually required
Latency	Very low (milliseconds)	Higher due to network communication
Privacy	Better, data stays on the device	Data is transmitted to remote servers
Power Consumption	Designed to be low	Typically much higher
Processing Power	Limited	Very high
Model Size	Small, optimized	Large and complex
Memory	KB to MB	GB to TB
Cost per Device	Low	Higher infrastructure costs
Scalability	Distributed across devices	Centralized infrastructure

Hardware Comparison

Embedded AI Devices	Traditional AI Platforms
ESP32-S3 DevKitC-1	Desktop PCs
STM32H7 Discovery Kit	GPU Workstations
Arduino Nano 33 BLE Sense	Cloud Servers
Raspberry Pi 5	Data Centers
NVIDIA Jetson Orin Nano Developer Kit	AI Supercomputers

Board	Typical Power	Typical Cost
ESP32 DevKit	0.1–1 W (much lower in deep sleep)	US\$4–8
ESP32-S3 DevKit	0.2–1.5 W	US\$8–20
Raspberry Pi 5 (8 GB)	5–15 W	US\$80–100
Jetson Orin Nano Developer Kit	7–25 W (configurable)	US\$250–500



AI Performance Comparison of Different Platforms

Platform	AI Performance	Memory	Power	Typical Cost	Typical AI Tasks
ESP32-S3	★☆☆☆☆	KB–MB	<1 W	Very low	TinyML, keyword spotting
Raspberry Pi 5	★★☆☆☆	4–8 GB	5–15 W	Low	OpenCV, TensorFlow Lite
Jetson Nano	★★★★☆	4 GB	5–10 W	Moderate (used)	Object detection
Jetson Orin Nano	★★★★☆	8–16 GB	7–25 W	Higher	Modern edge AI
Smartphone (2024–2026 flagship)	★★★★☆	8–24 GB	3–10 W	High	On-device AI, vision, speech
AI Laptop	★★★★☆	16–64 GB	20–100 W	High	ML development, inference
Gaming PC	★★★★★	32–128 GB	300–800 W	Very high	Training, LLMs, diffusion
AI Workstation	★★★★★	64 GB+	500–1500 W	Very high	Large-scale training
Cloud GPU	★★★★★	Hundreds of GB	Kilowatts	Usage-based	Foundation models

AI Performance Benchmarks

- **FLOPS (Floating Point Operations Per Second):**
 - FLOPS measures how many **floating-point arithmetic operations** a processor can execute in one second.
 - Floating-point numbers are real numbers such as 3.141592.
 - These are used extensively in Neural networks, Scientific computing, Signal processing, Image processing, Robotics, etc.
 - E.g., Suppose an image classification model requires 500 billion operations. A processor capable of 50 GFLOPS would theoretically require approximately $500 / 50 = 10$ seconds (assuming every operation is floating point and the processor is fully utilized).

Unit	Meaning	Operation s/second
KFLOPS	Thousand FLOPS	10^3
MFLOPS	Million FLOPS	10^6
GFLOPS	Billion FLOPS	10^9
TFLOPS	Trillion FLOPS	10^{12}
PFLOPS	Quadrillion FLOPS	10^{15}
EFLOPS	Quintillion FLOPS	10^{18}

AI Performance Benchmarks Cont.

- **TOPS (Tera Operations Per Second):**

- Unlike FLOPS, TOPS usually refers to INT8, INT4, and Mixed precision operations used in AI inference.
- Many trained neural networks can be converted from FP32 to INT8. Instead of 0.123456 they use 123 with an associated scaling factor.
- Advantages: Faster execution, Less memory, Lower power, Smaller models.

Data Type	Bits	Example
FP64	64	3.141592653589793
FP32	32	3.1415927
FP16	16	3.1406
INT8	8	127
INT4	4	7

Device	TOPS
Google Coral TPU	4
Hailo-8	26
Jetson Orin Nano Super	≈67
Modern smartphone NPU	30–80

AI Performance Benchmarks Cont.

- **Tensor TFLOPS:**

- Modern NVIDIA GPUs contain **Tensor Cores**.
- Tensor Cores perform matrix multiplication far more efficiently than conventional CUDA cores. This is why AI papers often quote FP16 Tensor TFLOPS rather than FP32 TFLOPS.

- **INT8 TOPS:**

- This is currently the most common benchmark for embedded AI.
- Example 67 INT8 TOPS means approximately 67 trillion INT8 operations every second assuming ideal conditions.

AI Performance Benchmarks Cont.

- **Memory Bandwidth:**

- Often overlooked, this metric is critical.
- Memory bandwidth measures how quickly data moves between memory and the processor. If the processor waits for data, high TOPS or TFLOPS cannot be fully utilized.
- Units: GB/s

- **Memory Capacity:**

- Neural networks occupy memory. This is why an ESP32-S3 cannot run an LLM, even if the arithmetic throughput were sufficient.

Device	Bandwidth
ESP32-S3	Very low
Raspberry Pi 5	~68 GB/s (system memory)
Jetson Orin Nano	>100 GB/s (LPDDR5)
RTX 4090	~1 TB/s

Model	Memory Needed
TinyML keyword spotter	<500 KB
MobileNet	Tens of MB
YOLO	Hundreds of MB
Small LLM	Several GB
Large LLM	Hundreds of GB

AI Performance Benchmarks Cont.

- **Latency:**

- Latency measures how long one inference takes.
- E.g., Camera → Face detector → Display result: If inference takes 20 ms the system feels real time. If it takes 1000 ms the experience is poor.

- **Throughput:**

- Throughput measures how much work can be completed per unit time.
- E.g., Images processed per second: YOLO 10 FPS, 30 FPS, 120 FPS, etc.

- **Inference Time:**

- Shorter inference time means faster decisions.

- **Energy Efficiency:**

- This is especially important for edge AI.
- Often expressed as TOPS/W or Images per Joule.

Standardized Benchmark Suites

- Instead of relying only on theoretical specifications, standardized benchmarks compare real-world performance. These benchmarks use actual workloads and are generally more informative than peak TOPS or TFLOPS alone.

Benchmark	Measures	Common Use
MLPerf	AI training and inference	Industry-standard AI benchmark
AI Benchmark	Smartphone AI performance	Mobile NPUs and CPUs
Geekbench AI	CPU, GPU, and NPU AI inference	Cross-platform comparison
SPEC CPU	General CPU performance	Scientific and engineering workloads
LINPACK	Floating-point performance	Supercomputers and HPC
Phoronix Test Suite	Linux system performance	SBCs and servers

Summary of Key Metrics

Metric	Measures	Best For	Important For
FLOPS	Floating-point arithmetic rate	CPUs, GPUs	Training, scientific computing
TFLOPS	Trillions of floating-point operations/s	GPUs	Deep learning compute
TOPS	Trillions of AI operations/s	NPU, TPUs	AI inference
Memory Bandwidth	Data transfer speed	All platforms	Large models, GPUs
RAM/VRAM	Model storage capacity	All platforms	Determines model size
Latency	Time for one inference	Robotics, real-time AI	User responsiveness
Throughput	Tasks completed per second	Servers, video analytics	System capacity
FPS	Frames processed per second	Computer vision	Real-time video
TOPS/W	AI performance per watt	Embedded and edge devices	Battery-powered systems
MLPerf / Geekbench AI	Standardized benchmark scores	Cross-platform comparisons	Real-world evaluation

Approximate AI Compute Comparison (2026)

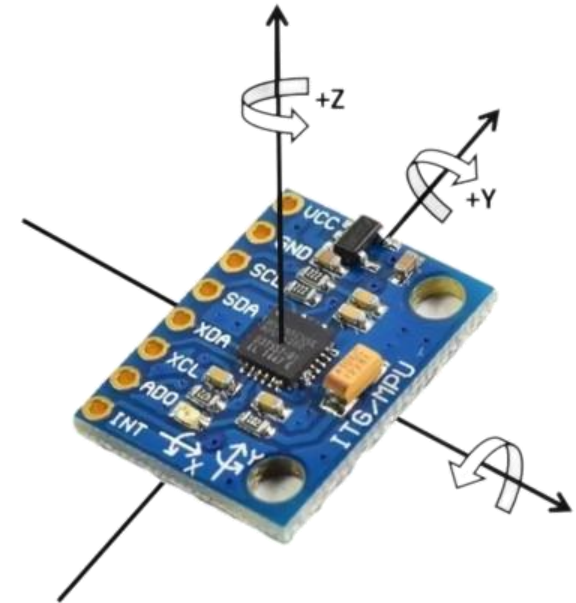
Platform	CPU	GPU	NPU/AI Accelerator	Approximate AI Compute
ESP32-S3	Dual Xtensa LX7	None	Vector instructions	Hundreds of MFLOPS
Raspberry Pi 5	Quad Cortex-A76	VideoCore VII	None	~20–40 GFLOPS (CPU)
Jetson Nano	Quad Cortex-A57	Maxwell GPU	None	~0.47 TFLOPS (FP16 GPU)
Jetson Orin Nano Super	6× Cortex-A78AE	Ampere GPU + Tensor Cores	Yes	≈67 TOPS (INT8)
Raspberry Pi 5 + Hailo-8	Cortex-A76	VideoCore VII	Hailo-8	26 TOPS
Raspberry Pi 5 + Hailo-10H	Cortex-A76	VideoCore VII	Hailo-10H	≈40 TOPS
Google Coral USB	Any host	—	Edge TPU	4 TOPS
Apple A18 Pro	6-core CPU	Apple GPU	Apple Neural Engine	≈35 TOPS
Qualcomm Snapdragon 8 Elite	Oryon CPU	Adreno GPU	Hexagon NPU	≈45–80 TOPS (vendor-dependent workload)
Intel Core Ultra 9 (Series 2)	Multi-core	Intel Arc	Intel AI Boost	≈100–120 TOPS (platform)
NVIDIA RTX 4060	CPU dependent	Ada GPU	Tensor Cores	~15 TFLOPS FP32; AI throughput is much higher with Tensor Cores
NVIDIA RTX 4090	CPU dependent	Ada GPU	Tensor Cores	~82 TFLOPS FP32
NVIDIA RTX 5090	CPU dependent	Blackwell GPU	Tensor Cores	>100 TFLOPS FP32 (generation dependent)
NVIDIA H100	Server CPU	Hopper GPU	Tensor Cores	~60 TFLOPS FP64; much higher at FP16/BF16/FP8

AI/ML Development for ESP32

- The **ESP32** is a low-cost microcontroller with integrated Wi-Fi and Bluetooth that is increasingly used for **Tiny Machine Learning (TinyML)** and **Edge AI** applications. Although it is not as powerful as a Raspberry Pi or NVIDIA Jetson, it can run **optimized AI models locally** with **low power consumption**, making it suitable for battery-powered IoT devices.
- What is TinyML?
 - **TinyML** is the deployment of machine learning models on low-power embedded devices such as: ESP32, Arduino Nano 33 BLE Sense, STM32, Nordic nRF52, RP2040.
- Note: the **training is almost always done on a PC or cloud GPU**, while the ESP32 performs **inference** (prediction).

AI Applications on ESP32

- 1. Voice Recognition
 - E.g., “Turn on light,” “Open door,” “Start motor”
 - Uses: ESP-SR, TensorFlow Lite Micro
- 2. Image Classification
 - Using an ESP32-CAM or ESP32-S3 with camera.
 - E.g., Cat vs Dog, Plant disease detection, QR detection, Object recognition
- 3. Gesture Recognition
 - Using MPU6050, ICM20948, BMI160
 - Recognize Wave, Shake, Circle, Tap

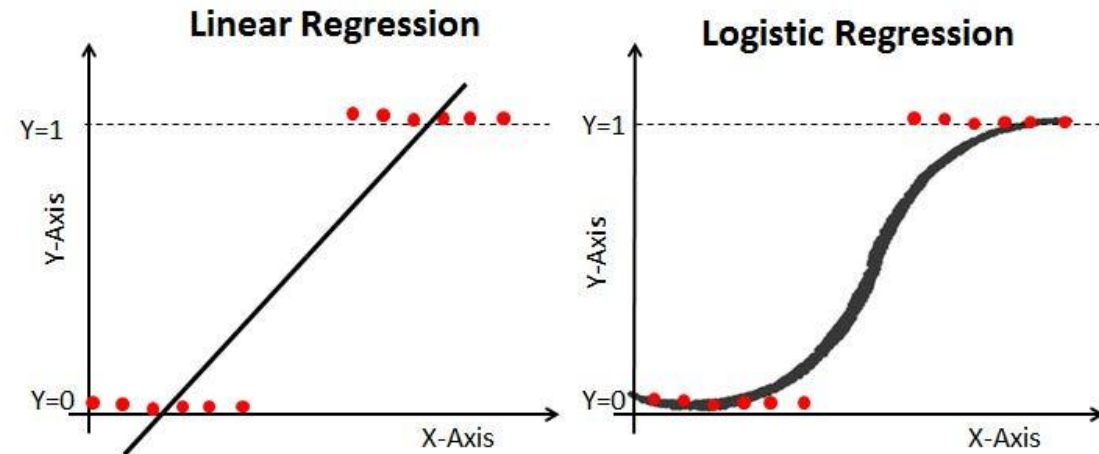


AI Applications on ESP32 Cont.

- 4. Predictive Maintenance
 - Sensors: Vibration, Current, Temperature
 - AI predicts: Bearing failure, Motor wear, Pump malfunction
- 5. Anomaly Detection
 - E.g., Normal vibration: □□□□□□ → Abnormal: □□□■□□■□□
 - The model detects unusual behavior before failure.
- 6. Human Activity Recognition
 - Recognize: Walking, Running, Sitting, Falling
 - Widely used in wearable devices.
- 7. Smart Agriculture
 - Predict: Irrigation needs, Plant stress, Soil conditions
 - Inputs: Moisture, Temperature, Humidity, Light
- 8. Smart Home
 - Detect: Occupancy, Voice commands, Appliance usage, Energy consumption

Machine Learning Models Suitable for ESP32

Model	Complexity	ESP32 Suitable
Linear Regression	Very Low	✓
Logistic Regression	Very Low	✓
Decision Tree	Low	✓
Random Forest (small)	Moderate	Limited
SVM	Moderate	Small models
K-Means	Low	✓
Neural Network (small)	Moderate	✓
CNN (small)	Moderate	ESP32-S3 preferred
RNN	High	Limited
Transformer	Very High	X



Popular AI Frameworks for ESP32

- 1. TensorFlow Lite for Microcontrollers
 - Most widely used.
 - Supports: Neural networks, CNN, Regression, Classification
 - Pros: Free, Open source, Large community
- 2. Edge Impulse
 - A cloud platform for TinyML. Excellent for beginners.
 - Features: Data collection, Labeling, Model training, Optimization, One-click deployment
- 3. ESP-DL
 - Developed by Espressif Systems. Optimized specifically for ESP32 and ESP32-S3.
 - Supports: CNN, Face detection, Image classification
- 4. ESP-SR
 - Speech recognition library.
 - Supports: Wake words, Command recognition, Noise reduction

Memory Requirements & Performance

Model Size	Flash	RAM
Tiny	20–50 KB	20 KB
Small	100 KB	60 KB
Medium	300 KB	150 KB
Large	1 MB+	ESP32-S3 with PSRAM (Pseudo-Static RAM)

Model	ESP32	ESP32-S3
Linear Regression	<1 ms	<1 ms
Small Neural Network	2–10 ms	1–5 ms
Gesture Recognition	5–20 ms	2–10 ms
Speech Command	50–200 ms	20–100 ms
Image Classification	300–1500 ms	50–300 ms

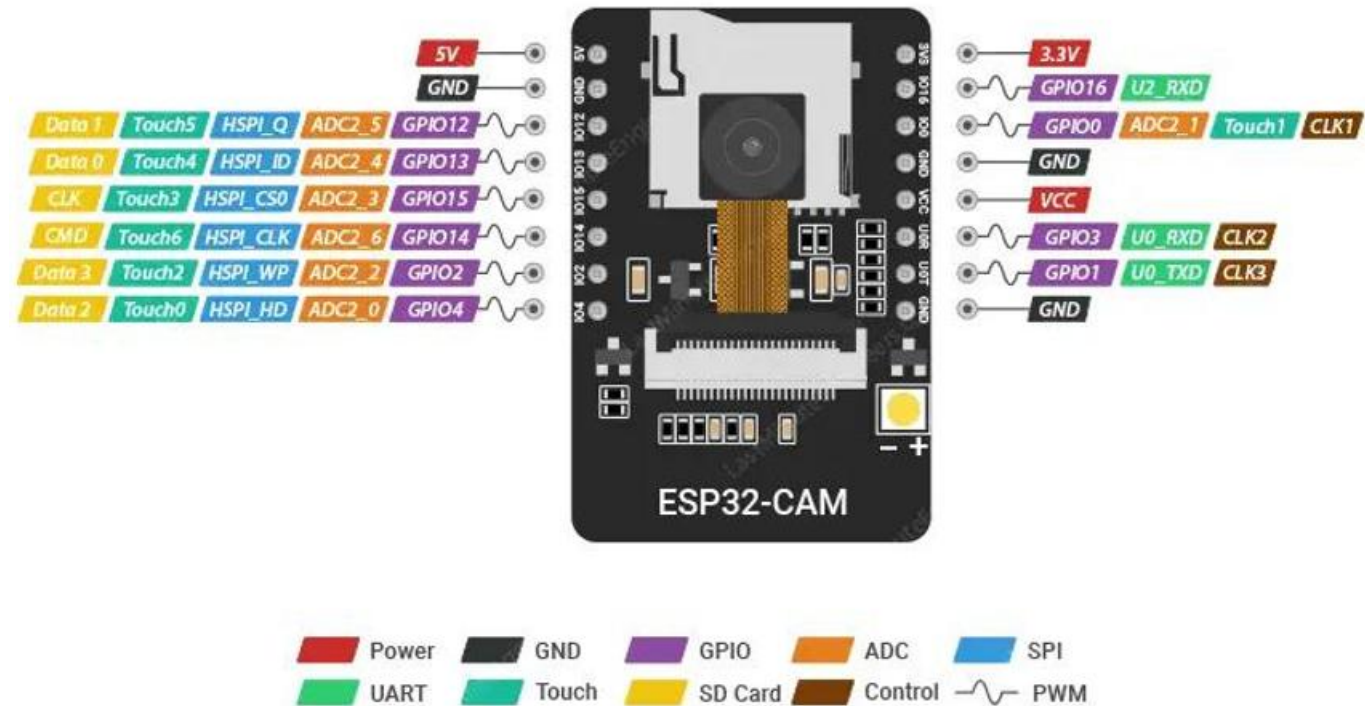
ESP32 vs ESP32-S3 for AI

Feature	ESP32	ESP32-S3
CPU	Dual-core Xtensa LX6	Dual-core Xtensa LX7
Vector Instructions	No	Yes (optimized for AI)
AI Performance	Good	Excellent
Camera Support	Good	Excellent
USB	No native USB	Native USB OTG
PSRAM Support	Optional	Better support
Recommended for TinyML	Basic models	Most TinyML projects

ESP32-CAM

ESP32-CAM

- The **ESP32-CAM** is a compact, low-cost development board that combines an **ESP32 microcontroller**, a **camera**, **Wi-Fi**, **Bluetooth**, and a **microSD card slot**. It is designed for applications involving image capture, video streaming, computer vision, and IoT.



ESP32-CAM Cont.

- The ESP32-CAM is based on the **ESP32-S** series microcontroller from Espressif Systems and is most commonly sold as the **AI-Thinker ESP32-CAM** module.
- It integrates:
 - Dual-core ESP32 processor
 - Wi-Fi (2.4 GHz)
 - Bluetooth Classic + BLE
 - OV2640 2 MP camera
 - microSD card interface
 - GPIO pins for sensors and actuators
 - Deep sleep capability for low-power applications
- Unlike the ESP32 DevKit boards, the ESP32-CAM **does not include a USB-to-Serial converter**, so an external programmer (or an Arduino Uno used as one) is required for uploading programs.

Hardware Specifications & Camera Resolutions

Feature	Specification
Processor	ESP32 Dual-Core Xtensa LX6
Clock Speed	Up to 240 MHz
SRAM	520 KB
PSRAM	Usually 4 MB (on AI-Thinker version)
Flash Memory	4 MB
Wi-Fi	802.11 b/g/n (2.4 GHz)
Bluetooth	Bluetooth Classic + BLE
Camera	OV2640, 2 MP
Image Resolution	Up to 1600 × 1200 (UXGA)
Storage	microSD (TF card)
Operating Voltage	5 V input (on-board regulator provides 3.3 V)

Resolution	Size
QQVGA	160 × 120
HQVGA	240 × 176
QVGA	320 × 240
CIF	352 × 288
VGA	640 × 480
SVGA	800 × 600
XGA	1024 × 768
SXGA	1280 × 1024
UXGA	1600 × 1200

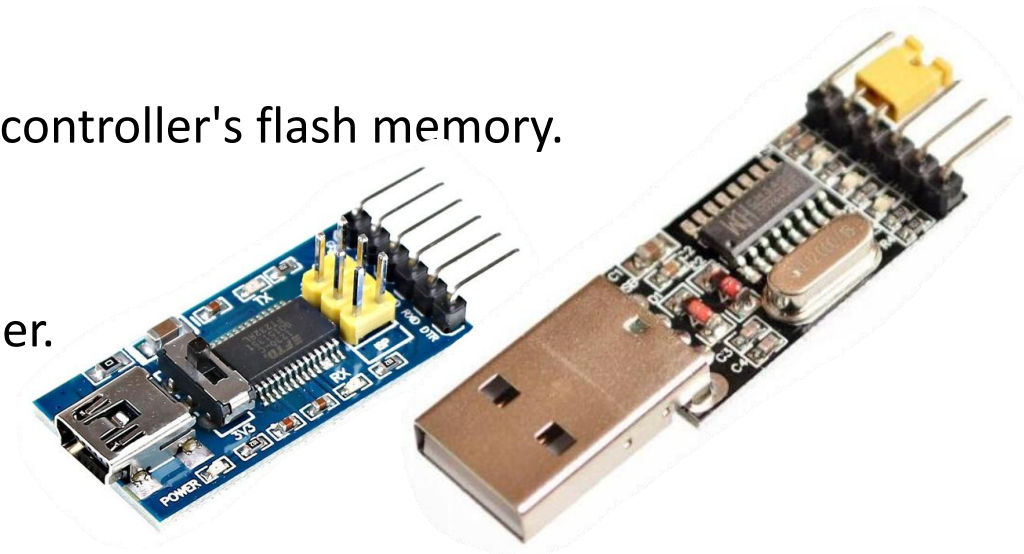
Typical Applications

- Wi-Fi security cameras
- Smart doorbells
- Face detection
- Face recognition
- QR code and barcode scanning
- Smart attendance systems
- License plate recognition (basic implementations)
- Wildlife monitoring
- Time-lapse photography
- Plant growth monitoring
- Remote image capture
- IoT monitoring with photos



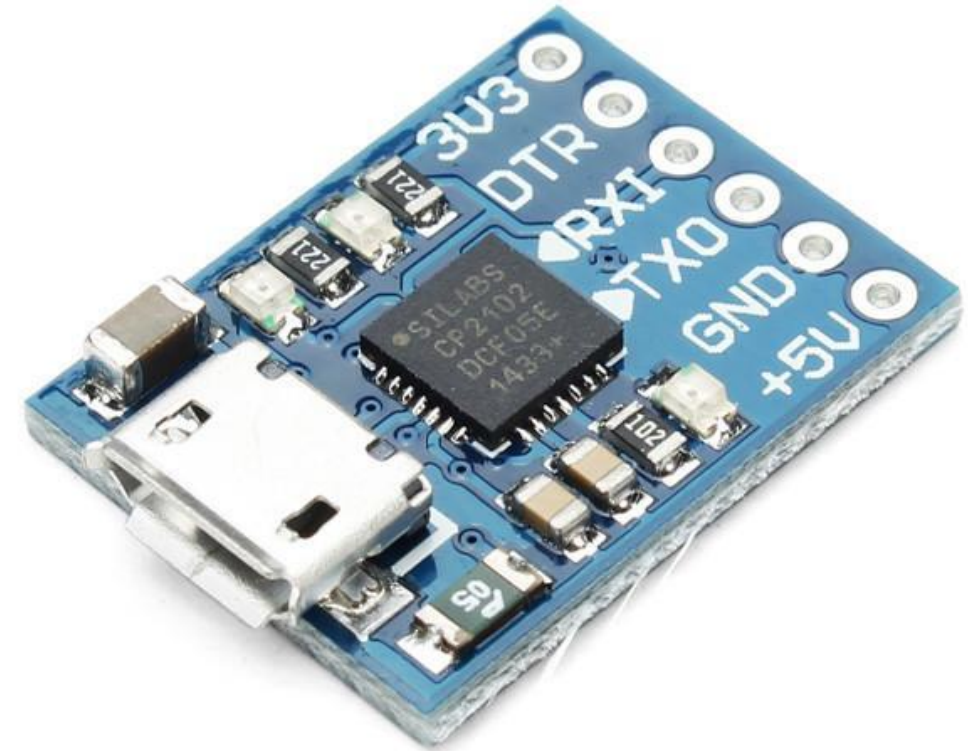
USB-to-Serial Programmer

- A **USB-to-Serial programmer** (also called a **USB-to-UART converter**, **USB-to-TTL adapter**, or **USB Serial Adapter**) is a device that allows a computer to communicate with the serial (UART) interface of a microcontroller. It is one of the most essential tools in embedded systems development.
- A USB-to-Serial programmer performs two main functions:
 - Program (flash) the microcontroller
 - Uploads firmware from the computer to the microcontroller's flash memory.
 - Serial communication
 - Displays debug messages in the Serial Monitor.
 - Sends commands from the PC to the microcontroller.
 - Receives sensor data or logs.



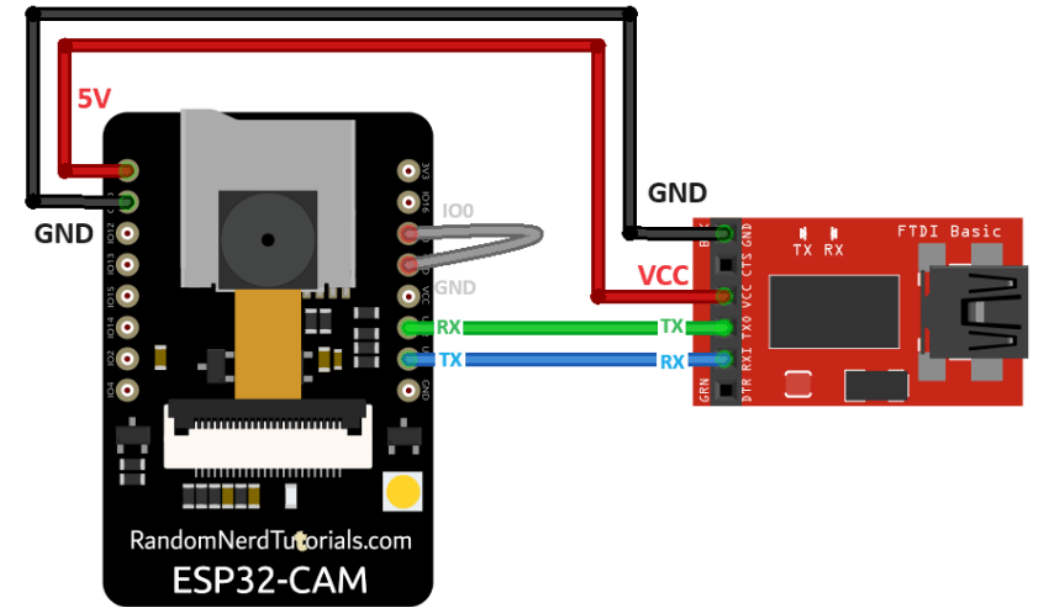
Common USB-to-Serial Chips

Chip	Manufacturer	Pros	Notes
CP2102	Silicon Labs	Very reliable	Common with ESP32 boards
CP2104	Silicon Labs	Newer version	Stable drivers
CH340G	WCH	Very inexpensive	Common on Arduino clones
FT232RL	FTDI	High quality	More expensive
PL2303	Prolific	Older design	Less common today



Programming an ESP32

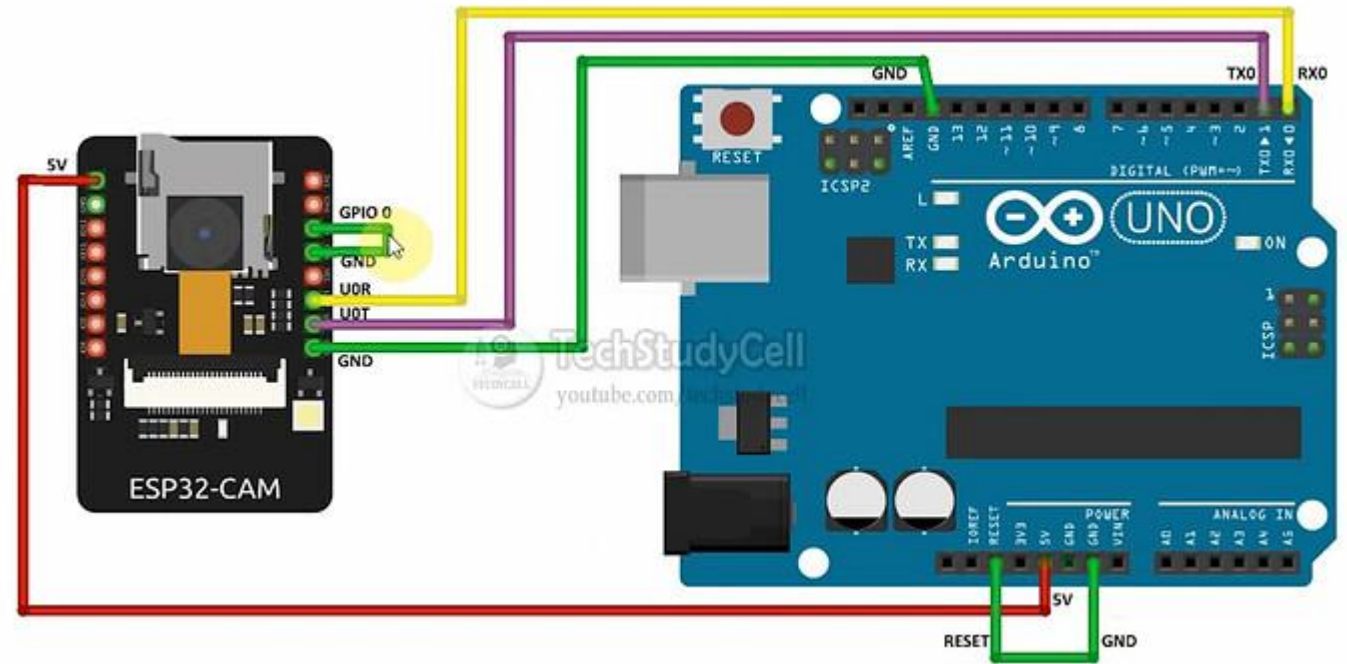
- Most ESP32 development boards are powered from the **5V/VIN** pin because they include an onboard 3.3 V regulator. Bare ESP32 modules typically require a regulated 3.3 V supply.
- During programming, **GPIO0** is held low (connected to GND) and the board is reset to enter the bootloader.



USB Adapter	ESP32
TX	RX0 (GPIO3)
RX	TX0 (GPIO1)
GND	GND
5V or 3.3V*	VIN/5V or 3.3V (depending on the board)

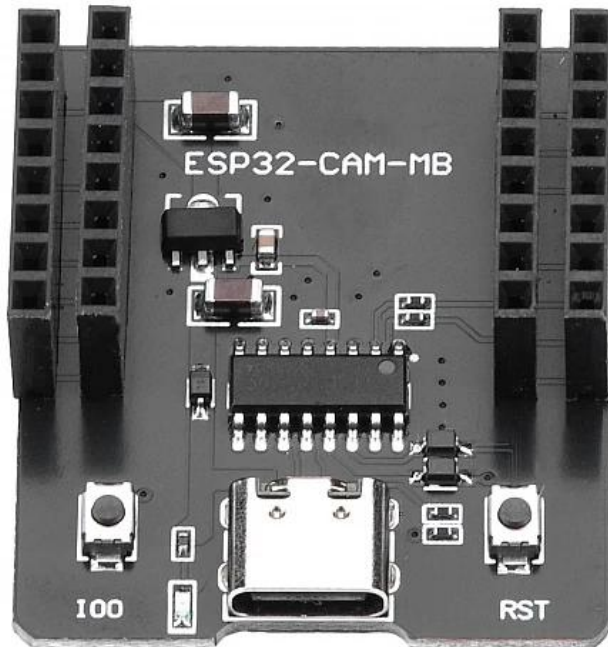
Arduino Uno R3 as a USB-to-Serial Programmer

- The Uno outputs 5 V logic while the ESP32 UART pins are 3.3 V. Many hobbyists use this wiring successfully, but it is outside the ESP32's recommended logic levels. A proper 3.3 V USB-to-TTL adapter or a level shifter is safer for long-term use.



ESP32 CAM Adapter Board

- The recommended approach.



Project 1: Blink the onboard flash LED

- **Step 1: Open VS Code**

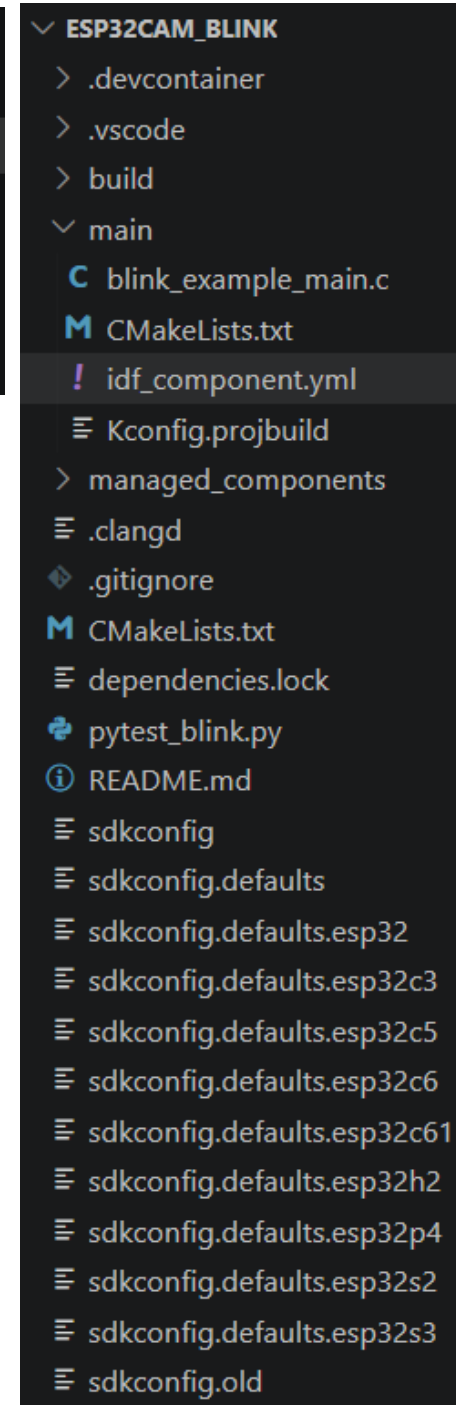
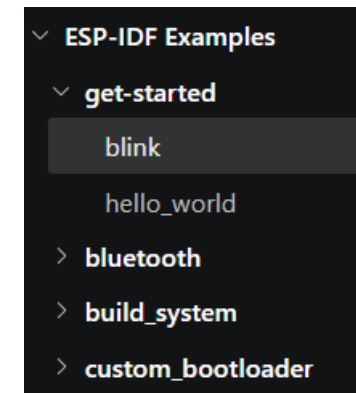
- If the ESP-IDF extension is installed (e.g., v2.1.0), you should see an **Espressif** icon in the Activity Bar on the left.

- **Step 2: Verify the ESP-IDF installation**

- Press **Ctrl + Shift + P** (or from commands) and run: **ESP-IDF: Doctor Command**
- If everything is installed correctly, you should see messages similar to: Python OK, ESP-IDF OK, CMake OK, Ninja OK, OpenOCD OK...

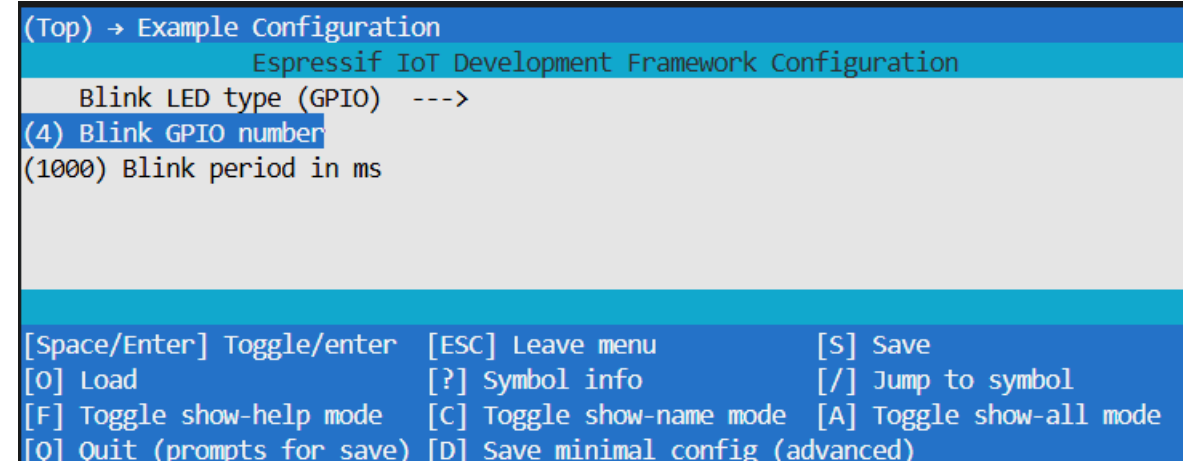
- **Step 3: Create a New Project**

- Run: **ESP-IDF: New Project**
- Choose: Template: Blink, Project Name: ESP32CAM_Blink, Location: Your preferred folder (for example, D:\ESP\Projects), Target: ESP32, Serial Port, OpenOCD Configuration: board/esp32-wrover-kit-3.3v.cfg (leave as it is)



Project 1 Cont.

- **Step 4: Change the Blink GPIO**
 - In the ESP-IDF Terminal, run: **idf.py menuconfig**
 - Under Example Configuration, change the Blink GPIO number to 4
- **Step 5: Build the Project**
 - Run: **ESP-IDF: Build your project**
 - Or, In the ESP-IDF Terminal, run: **idf.py build**, or click the **Build Icon** from menu
- **Step 6: Flash the Project**
 - Run: **ESP-IDF: Flash your project**
 - Or, In the ESP-IDF Terminal, run: **idf.py -p COM5 -b 115200 flash**, or click the **Build Icon** from menu



```
(Top) → Example Configuration
Espressif IoT Development Framework Configuration
Blink LED type (GPIO) --->
(4) Blink GPIO number
(1000) Blink period in ms

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                  [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Project 2: Use the OV2640 Camera to Capture a single JPEG image

- **Step 1 – Create a New Project**

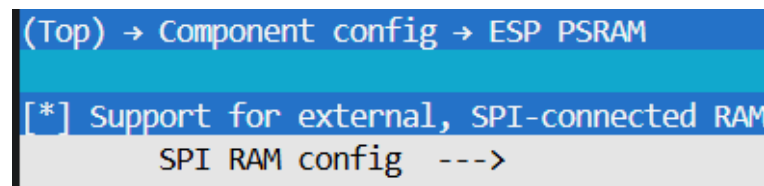
- Create a new ESP-IDF project (use the same blink template):
ESP32CAM_Capture

- **Step 2 – Add the Camera Component**

- From the ESP-IDF Terminal: **idf.py add-dependency “espressif/esp32-camera”**

- **Step 3 – Enable PSRAM**

- The camera driver expects PSRAM for most practical image sizes.
- Open: **idf.py menuconfig** → Component config → ESP PSRAM: Enable Support for external SPI RAM



Project 2 Cont.

- **Step 4: Replace `blink_example_main.c`**
- **Step 5: Build the Project**
 - Run: **ESP-IDF: Build your project**
 - Or, In the ESP-IDF Terminal, run: **`idf.py build`**, or click the **Build Icon** from menu
- **Step 6: Flash the Project**
 - Run: **ESP-IDF: Flash your project**
 - Or, In the ESP-IDF Terminal, run: **`idf.py -p COM5 -b 115200 flash`**, or click the **Build Icon** from menu
- **Step 7: Monitor**
 - Run: **ESP-IDF: Monitor your device**

```
I (216573) CAMERA: JPEG Captured
I (216573) CAMERA: Width : 640
I (216573) CAMERA: Height : 480
I (216573) CAMERA: Length : 14018 bytes
I (216573) CAMERA: Buffer : 0x3f800dd0
I (216573) CAMERA: Frame Buffer Returned
```

Project 2 Main

```
1  #include <stdio.h>
2
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5
6  #include "esp_camera.h"
7  #include "esp_log.h"
8
9  static const char *TAG = "CAMERA";
10
11 /* AI Thinker ESP32-CAM pin definitions */
12
13 #define CAM_PIN_PWDN 32
14 #define CAM_PIN_RESET -1
15 #define CAM_PIN_XCLK 0
16
17 #define CAM_PIN_SIOD 26
18 #define CAM_PIN_SIOC 27
19
20 #define CAM_PIN_D7 35
21 #define CAM_PIN_D6 34
22 #define CAM_PIN_D5 39
23 #define CAM_PIN_D4 36
24 #define CAM_PIN_D3 21
25 #define CAM_PIN_D2 19
26 #define CAM_PIN_D1 18
27 #define CAM_PIN_D0 5
28
29 #define CAM_PIN_VSYNC 25
30 #define CAM_PIN_HREF 23
31 #define CAM_PIN_PCLK 22
32
33 void app_main(void)
34 {
35     camera_config_t config = {
36
37         .pin_pwdn = CAM_PIN_PWDN,
38         .pin_reset = CAM_PIN_RESET,
39         .pin_xclk = CAM_PIN_XCLK,
40
41         .pin_sccb_sda = CAM_PIN_SIOD,
42         .pin_sccb_scl = CAM_PIN_SIOC,
43
44         .pin_d7 = CAM_PIN_D7,
45         .pin_d6 = CAM_PIN_D6,
46         .pin_d5 = CAM_PIN_D5,
47         .pin_d4 = CAM_PIN_D4,
48         .pin_d3 = CAM_PIN_D3,
49         .pin_d2 = CAM_PIN_D2,
50         .pin_d1 = CAM_PIN_D1,
51         .pin_d0 = CAM_PIN_D0,
52
53         .pin_vsync = CAM_PIN_VSYNC,
54         .pin_href = CAM_PIN_HREF,
55         .pin_pclk = CAM_PIN_PCLK,
56
57         .xclk_freq_hz = 20000000,
58
59         .ledc_timer = LEDC_TIMER_0,
60         .ledc_channel = LEDC_CHANNEL_0,
61
62         .pixel_format = PIXFORMAT_JPEG,
63
64         .frame_size = FRAMESIZE_VGA,
65
66         .jpeg_quality = 12,
67
68         .fb_count = 1,
69
70         .grab_mode = CAMERA_GRAB_WHEN_EMPTY
71     };
72
73     esp_err_t err = esp_camera_init(&config);
74
75     if (err != ESP_OK)
76     {
77         ESP_LOGE(TAG,
78             "Camera Init Failed: 0x%x",
79             err);
80
81         return;
82     }
83
84     ESP_LOGI(TAG,
85         "Camera initialized successfully");
86
87     while (1)
88     {
89         camera_fb_t *fb = esp_camera_fb_get();
90
91         if (!fb)
92         {
93             ESP_LOGE(TAG,
94                 "Failed to capture image");
95
96             vTaskDelay(pdMS_TO_TICKS(1000));
97
98             continue;
99         }
100
101         ESP_LOGI(TAG,
102             "JPEG Captured");
103
104         ESP_LOGI(TAG,
105             "Width : %d",
106             fb->width);
107
108         ESP_LOGI(TAG,
109             "Height : %d",
110             fb->height);
111
112         ESP_LOGI(TAG,
113             "Length : %d bytes",
114             fb->len);
115
116         ESP_LOGI(TAG,
117             "Buffer : %p",
118             fb->buf);
119
120         esp_camera_fb_return(fb);
121
122         ESP_LOGI(TAG,
123             "Frame Buffer Returned");
124
125         vTaskDelay(pdMS_TO_TICKS(5000));
126     }
127 }
```

Project 3: SD Card Read/Write Text

```
#include <stdio.h>
#include <string.h>

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "esp_log.h"
#include "esp_vfs_fat.h"

#include "driver/sdmmc_host.h"
#include "sdmmc_cmd.h"

static const char *TAG = "SDCARD";

#define MOUNT_POINT "/sdcard"

void app_main(void)
{
    esp_err_t ret;

    sdmmc_host_t host = SDMMC_HOST_DEFAULT();

    // 1-bit mode (recommended for AI Thinker ESP32-CAM)
    sdmmc_slot_config_t slot_config = SDMMC_SLOT_CONFIG_DEFAULT();
    slot_config.width = 1;

    esp_vfs_fat_sdmmc_mount_config_t mount_config = {
        .format_if_mount_failed = false,
        .max_files = 5,
        .allocation_unit_size = 16 * 1024
    };

    sdmmc_card_t *card;

    ESP_LOGI(TAG, "Mounting SD card...");
```

```
    ret = esp_vfs_fat_sdmmc_mount(
        MOUNT_POINT,
        &host,
        &slot_config,
        &mount_config,
        &card);

    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Mount failed: %s", esp_err_to_name(ret));
        return;
    }

    ESP_LOGI(TAG, "SD Card Mounted");
    sdmmc_card_print_info(stdout, card);

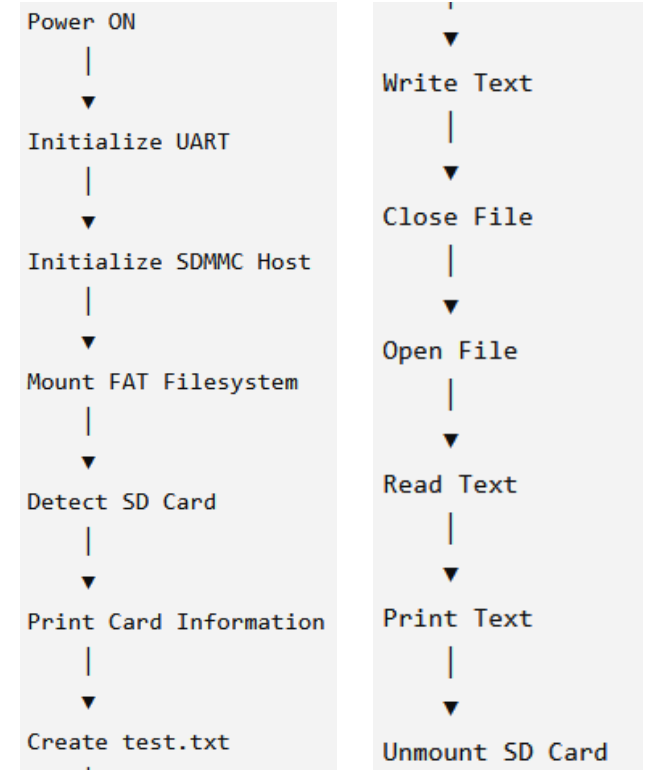
    FILE *f = fopen("/sdcard/test.txt", "w");
    if (!f) {
        ESP_LOGE(TAG, "Failed to create file");
        esp_vfs_fat_sdcard_unmount(MOUNT_POINT, card);
        return;
    }

    fprintf(f, "Hello ESP32-CAM!\n");
    fprintf(f, "Project 5A - SDMMC + FAT Filesystem\n");
    fprintf(f, "ESP-IDF 5.5.4\n");
    fclose(f);

    ESP_LOGI(TAG, "File written.");

    char line[128];

    f = fopen("/sdcard/test.txt", "r");
    if (!f) {
        ESP_LOGE(TAG, "Failed to open file");
        esp_vfs_fat_sdcard_unmount(MOUNT_POINT, card);
        return;
    }
```



```
    ESP_LOGI(TAG, "Reading file:");

    while (fgets(line, sizeof(line), f)) {
        printf("%s", line);
    }

    fclose(f);

    esp_vfs_fat_sdcard_unmount(MOUNT_POINT, card);

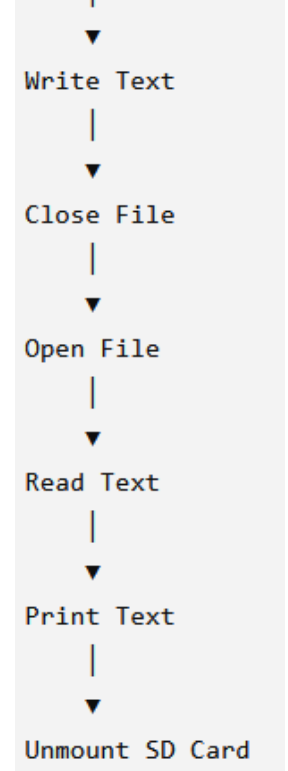
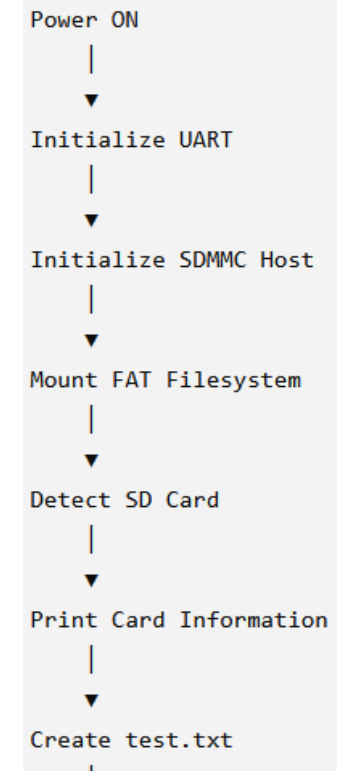
    ESP_LOGI(TAG, "SD card unmounted.");

    while (1) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
```

Project 3: SD Card Read/Write Text

The API Sequence

Function	Purpose
SDMMC_HOST_DEFAULT()	Configure the SDMMC host controller
SDMMC_SLOT_CONFIG_DEFAULT()	Configure the SDMMC slot and GPIOs
esp_vfs_fat_sdmmc_mount()	Mount the FAT filesystem on the SD card
sdmmc_card_print_info()	Display card information (name, capacity, etc.)
fopen()	Create or open a file
fprintf()	Write text to the file
fgets()	Read text from the file
fclose()	Close the file and flush data
esp_vfs_fat_sdcard_unmount()	Unmount the filesystem safely



Project 4: SD Card Read/Write Binary Data

```
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "esp_vfs_fat.h"
#include "driver/sdmmc_host.h"
#include "sdmmc_cmd.h"

#define MOUNT_POINT "/sdcard"
#define FILE_NAME "/sdcard/data.bin"
static const char *TAG="P5B";

static esp_err_t mount_card(sdmmc_card_t **card){
    sdmmc_host_t host=SDMMC_HOST_DEFAULT();
    sdmmc_slot_config_t slot=SDMMC_SLOT_CONFIG_DEFAULT();
    slot.width=1;
    esp_vfs_fat_sdmmc_mount_config_t cfg={
        .format_if_mount_failed=false,
        .max_files=5,
        .allocation_unit_size=16*1024
    };
    return esp_vfs_fat_sdmmc_mount(MOUNT_POINT,&host,&slot,&cfg,card);
}

void app_main(void){
    sdmmc_card_t *card=NULL;
    ESP_ERROR_CHECK(mount_card(&card));
    sdmmc_card_print_info(stdout,card);

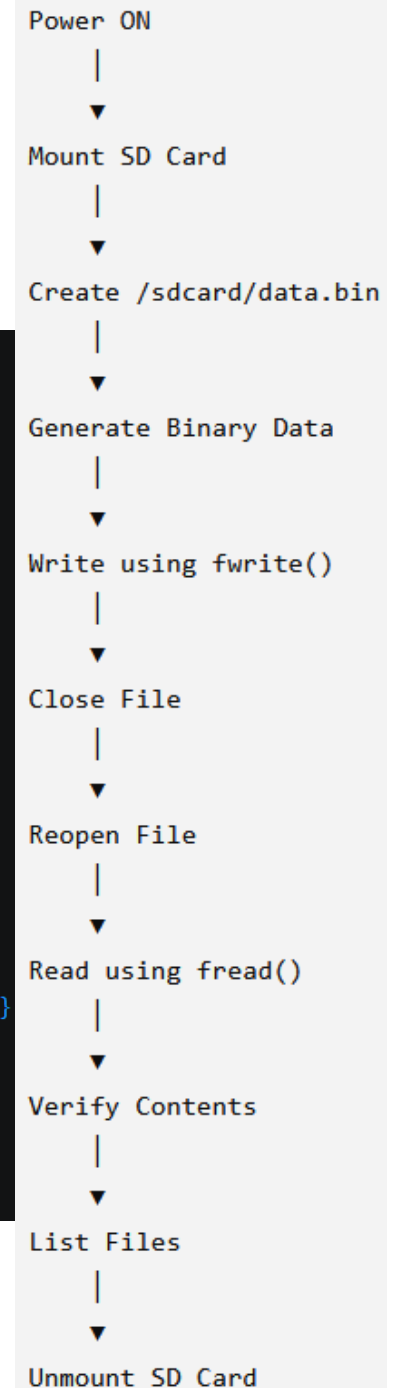
    uint8_t tx[256],rx[256];
    for(int i=0;i<256;i++) tx[i]=i;

    FILE *f=fopen(FILE_NAME,"wb");
    if(!f){ESP_LOGE(TAG,"open failed"); return;}
    fwrite(tx,1,sizeof(tx),f);
    fclose(f);

    memset(rx,0,sizeof(rx));
    f=fopen(FILE_NAME,"rb");
    if(!f){ESP_LOGE(TAG,"read open failed"); return;}
    fread(rx,1,sizeof(rx),f);
    fclose(f);

    bool ok=true;
    for(int i=0;i<256;i++){
        if(tx[i]!=rx[i]){ok=false; ESP_LOGE(TAG,"Mismatch at %d",i); break;}
    }
    if(ok) ESP_LOGI(TAG,"Binary verification PASSED");

    esp_vfs_fat_sdcard_unmount(MOUNT_POINT,card);
    while(1){vTaskDelay(pdMS_TO_TICKS(1000));}
}
```



Project 5: Capture Image and Save JPEG in SD Card

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_camera.h"
#include "esp_log.h"
#include "esp_vfs_fat.h"
#include "driver/sdmmc_host.h"
#include "sdmmc_cmd.h"
#define MOUNT_POINT "/sdcard"
#define CAM_PIN_PWDN 32
#define CAM_PIN_RESET -1
#define CAM_PIN_XCLK 0
#define CAM_PIN_SIOD 26
#define CAM_PIN_SIOC 27
#define CAM_PIN_D7 35
#define CAM_PIN_D6 34
#define CAM_PIN_D5 39
#define CAM_PIN_D4 36
#define CAM_PIN_D3 21
#define CAM_PIN_D2 19
#define CAM_PIN_D1 18
#define CAM_PIN_D0 5
#define CAM_PIN_VSYNC 25
#define CAM_PIN_HREF 23
#define CAM_PIN_PCLK 22

static esp_err_t cam_init(void){
    camera_config_t c={
        .pin_pwdn=CAM_PIN_PWDN,.pin_reset=CAM_PIN_RESET,.pin_xclk=CAM_PIN_XCLK,
        .pin_sccb_sda=CAM_PIN_SIOD,.pin_sccb_scl=CAM_PIN_SIOC,
        .pin_d7=CAM_PIN_D7,.pin_d6=CAM_PIN_D6,.pin_d5=CAM_PIN_D5,.pin_d4=CAM_PIN_D4,
        .pin_d3=CAM_PIN_D3,.pin_d2=CAM_PIN_D2,.pin_d1=CAM_PIN_D1,.pin_d0=CAM_PIN_D0,
        .pin_vsync=CAM_PIN_VSYNC,.pin_href=CAM_PIN_HREF,.pin_pclk=CAM_PIN_PCLK,
        .xclk_freq_hz=20000000,.ledc_timer=LED_TIMER_0,.ledc_channel=LED_CHANNEL_0,
        .xclk_freq_hz=20000000,.ledc_timer=LED_TIMER_0,.ledc_channel=LED_CHANNEL_0,
        .pixel_format=PIXFORMAT_JPEG,.frame_size=FRAMESIZE_VGA,.jpeg_quality=12,.fb_count=1,.grab_mode=CAMERA_GRAB_WHEN_EMPTY};
    return esp_camera_init(&c);
}

void app_main(void){
    sdmmc_host_t host=SDMMC_HOST_DEFAULT();
    sdmmc_slot_config_t slot=SDMMC_SLOT_CONFIG_DEFAULT(); slot.width=1;
    esp_vfs_fat_sdmmc_mount_config_t cfg={.format_if_mount_failed=false,.max_files=5,.allocation_unit_size=16384};
    sdmmc_card_t *card=NULL;
    ESP_ERROR_CHECK(cam_init());
    ESP_ERROR_CHECK(esp_vfs_fat_sdmmc_mount(MOUNT_POINT,&host,&slot,&cfg,&card));
    camera_fb_t *fb=esp_camera_fb_get();
    FILE *f=fopen("/sdcard/image001.jpg","wb");
    if(fb&&f){
        fwrite(fb->buf,1,fb->len,f); fclose(f); esp_log_level_set("",ESP_LOG_INFO);
        printf("Saved image001.jpg (%u bytes)\n",(unsigned)fb->len);
    }
    if(fb) esp_camera_fb_return(fb);
    esp_vfs_fat_sdcard_unmount(MOUNT_POINT,card);
    while(1){vTaskDelay(pdMS_TO_TICKS(1000));}
}
```

